

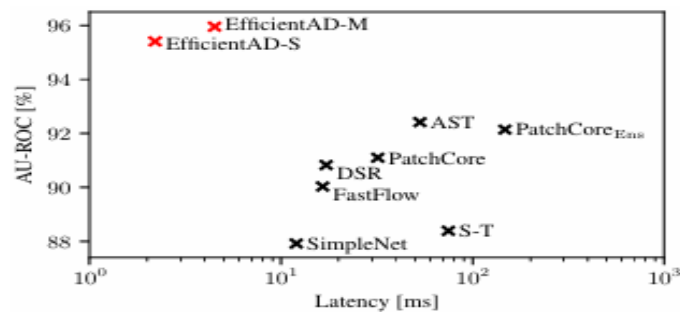


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش پژوهش درس شبکه‌های عصبی

A.K EfficientAD



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،

Aminkianiworkeng@gmail.com

استاد درس: دکتر برادران

نیمسال اول تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

۳	مستندات.....
۳	۱- خوانش مقاله.....
۴	۲- صحت‌سنجی نتایج (پیاده‌سازی).....
۶	۳- گزارش ایده.....
۸	۴- پیاده‌سازی.....
۹	۵- ارائه.....
۱۱	۶- مراجع.....

مستندات

۱- خوانش مقاله

انتخاب مقاله پایه: EfficientAD (انتشار در WACV 2024) به دلیل عملکرد برتر و کارایی بالاتر نسبت به SimpleNet برگزیده شده و این روش نه تنها دقت تشخیص ناهنجاری و مکان‌یابی نقص‌ها را ارتقاء داده، بلکه با زمان تأخیر در حد میلی‌ثانیه (حدود ۲ میلی‌ثانیه) و توان پردازش صدها تصویر در ثانیه به نیازهای بلادرنگ صنعت پاسخ می‌دهد. در مقابل، SimpleNet عملکرد ضعیف‌تر و سرعت پایین‌تری داشته است (برای مثال AU-ROC تصویر در SimpleNet حدود ۸۷.۹٪ در برابر ۹۵٪+ در EfficientAD، با تأخیر ۱۲ms در برابر ۲ms).

مشکلات حل‌شده توسط مدل: روش‌های پیشین تشخیص ناهنجاری معمولاً با یکی از دو مشکل اصلی مواجه بودند:

(۱) هزینه محاسباتی بالا و تأخیر زیاد برای رسیدن به دقت مناسب،

(۲) ناتوانی در تشخیص ناهنجاری‌های منطقی (مانند وجود یا نبود اجزای صحیح با چینش نادرست).

EfficientAD این مشکلات را با طراحی یک معماری بسیار سریع و سبک حل کرده و بدون استفاده از شبکه‌های سنگین یا ensemble، هم دقت بالا و هم سرعت بسیار زیاد را فراهم کرده است. همچنین با اضافه کردن یک مؤلفه سراسری، توانایی تشخیص ناهنجاری‌های منطقی را نیز به مدل افزوده است.

نحوه عملکرد روش پیشنهادی: EfficientAD مبتنی بر یک چارچوب معلم-شاگرد است. یک شبکه‌ی معلم سبک، ویژگی‌های تصویر را استخراج می‌کند و شبکه‌ی شاگرد فقط با تصاویر نرمال آموزش می‌بیند تا خروجی معلم را تقلید کند. در زمان آزمون، اختلاف بین خروجی معلم و شاگرد به‌عنوان نقشه‌ی ناهنجاری موضعی در نظر گرفته می‌شود. برای تشخیص ناهنجاری‌های منطقی، یک خودرمزگذار سراسری نیز به مدل اضافه شده که روابط کلی تصویر نرمال را یاد می‌گیرد. اختلاف بین خروجی خودرمزگذار و شاگرد، ناهنجاری‌های غیرموضعی را آشکار می‌کند. در نهایت، نقشه‌های ناهنجاری موضعی و سراسری پس از نرمال‌سازی مناسب با هم ترکیب می‌شوند.

کامپوننت‌های مؤثر مقاله در حل مسائل پیشین:

- استخراج‌کننده ویژگی بسیار سریع: باعث کاهش شدید تأخیر زمانی و امکان پردازش بلادرنگ شده است.
- تابع هزینه **Hard Feature Mining**: تمرکز آموزش روی نواحی دشوار و جلوگیری از یادگیری بیش‌ازحد شاگرد.
- جریمه‌ی داده‌های خارج از توزیع: مانع تعمیم ناخواسته‌ی مدل به تصاویر غیرنرمال می‌شود.
- خودرمزگذار سراسری: ضعف روش‌های موضعی در تشخیص ناهنجاری‌های منطقی را جبران کرده است.

عملکرد مدل در benchmarkهای مختلف: روی مجموعه داده‌های VisA، MVTec AD و LOCO MVTec ارزیابی شده و در همه‌ی آن‌ها عملکردی در سطح SOTA یا نزدیک به آن داشته است. این مدل در سطح تصویر به AUROC بالاتر از ۹۵٪ و در مکان‌یابی پیکسلی به نتایج بسیار دقیق دست یافته است. مهم‌تر از همه، زمان پردازش هر تصویر حدود ۲ تا ۴ میلی‌ثانیه گزارش شده که به مراتب سریع‌تر از روش‌هایی مانند SimpleNet، PatchCore و FastFlow است، بدون آن‌که دقت کاهش یابد.

۲- صحت‌سنجی نتایج (پیاده‌سازی)

بر اساس جستجوها و منابع موجود، تاکنون کد رسمی از طرف نویسندگان منتشر نشده‌است ولی یک پیاده‌سازی غیررسمی توسط جامعه پژوهشی ارائه شده است. مهم‌ترین منبع در این زمینه مخزن گیت‌هاب **EfficientAD** است که توسط کاربری به نام **nelson1425** ایجاد شده و تحت لیسانس **Apache-2.0** منتشر شده است.^۱ این مخزن پیاده‌سازی کامل روش **EfficientAD** را شامل می‌شود و نتایج گزارش شده آن نیز با نتایج مقاله تقریباً مطابقت دارد. همچنین این پیاده‌سازی غیررسمی در کتابخانه‌ی **Anomalib** ادغام شده است و توسعه‌دهندگان آن (از جمله **nelson1425** و **alexriedel1**) در نگارش کد مشارکت داشته‌اند.^۲ به طور خلاصه:

- **کد رسمی:** تاکنون منتشر نشده است (و به گفته‌ی منابع، کد اصلی توسط نویسندگان در دسترس عموم قرار نگرفته است).
- **کد غیررسمی:** در گیت‌هاب موجود است. دو مخزن برجسته عبارت‌اند از:

- مخزن اصلی غیررسمی: **nelson1425/EfficientAD** که پیاده‌سازی کامل مدل **EfficientAD** را ارائه می‌دهد. این مخزن ۳۸۲ ستاره دارد و توسط جامعه به رسمیت شناخته شده است.
- یک مخزن دیگر: **rximg/EfficientAD** که فورکی از همین پروژه یا نسخه‌ای مشابه است. این نسخه نیز نتایج بازتولیدشده‌ای را گزارش کرده و وزن‌های پیش‌آموزش‌یافته (**pretrained weights**) مدل را در بخش **Releases** خود قرار داده است.

با توجه به این که کد غیررسمی موجود است، نیازی به پیاده‌سازی صفر تا صد از ابتدا نیست در عوض می‌توان از پیاده‌سازی منتشرشده بهره گرفت و آن را برای اجرای در محیط گوگل کولب آماده‌سازی کرد.

قیاس نتایج:

«اعداد مقاله معمولاً میانگین روی تمام کلاس‌های **benchmark** هستند؛ اما در این پیاده‌سازی به دلیل محدودیت دانلود در **Colab**، فقط روی کلاس‌های موجود (مثلاً **bottle** و **breakfast_box**) اجرا شده و میانگین روی همین subset گزارش شده است. فرایند آموزش شامل **Teacher-Student learning** به همراه **AutoEncoder** بوده و **loss**های کلیدی (**loss_total, loss_st, loss_ae, loss_stae** و ...) به صورت مرحله‌ای در **TensorBoard** ثبت شدند. سپس در فاز آزمون، متریک اصلی مقاله یعنی **Image-level AUROC** محاسبه و در **TensorBoard** گزارش شد. برای متریک‌های مکانیابی (**Pixel AUROC** و **AU-PRO@0.3**) نیز یک ارزیاب سفارشی و کم‌مصرف از نظر حافظه پیاده‌سازی و نتایج در **TensorBoard** ثبت گردید. علاوه بر این، وزن‌های نهایی **student/autoencoder/teacher** برای هر دیتاست ذخیره و گزارش شدند.»

^۱ <https://github.com/nelson1425/EfficientAD>

^۲ <https://github.com/open-edge-platform/anomalib/discussions/1183>

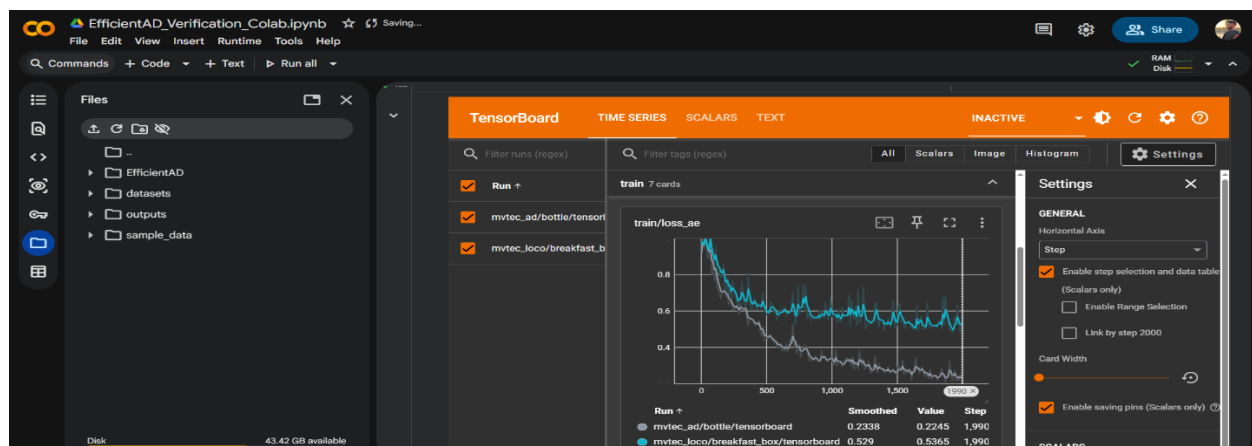
^۳ <https://github.com/rximg/EfficientAD>

آماده‌سازی کد در Google Colab:

Lossها (توابع خطا)

- train/loss_total : مجموع کل خطا (Total Loss)
 - train/loss_st : خطای Teacher-Student (بخش local / feature imitation)
 - train/loss_hard : بخش Hard Feature Mining (تمرکز روی سخت‌ترین پیکسل‌ها/ویژگی‌ها)
 - train/loss_penalty : جریمه‌ی OOD/penalty (اگر Imagenette فعال باشد)
 - train/loss_ae : خطای AutoEncoder (بخش global)
 - train/loss_stae : خطای Student-AE consistency
- Image-level AUROC : تشخیص ناهنجاری در سطح تصویر که مقدار نهایی به صورت زیر در فایل‌های خروجی ثبت شده است:
- MVTec AD / bottle: final_image_auc = 99.9206
 - MVTec LOCO / breakfast_box: final_image_auc = 84.0474
- از آن‌جا که اسکریپت رسمی eval در repo موجود نبود/لینک‌ها مشکل داشت، ما متریک‌های پیکسلی را با سلول ۹.۵ خودمان محاسبه کردیم و سپس داخل همان TensorBoard هر اجرا log کردیم:

- Pixel-level AUROC : برای localization
 - AU-PRO@0.3 : متریک رایج برای segmentation-like localization
- نکته : در LOCO ممکن است ground-truth پیکسلی برای بعضی anomaly ها (خصوصاً logical) وجود نداشته باشد یا الگوی نام‌گذاری ماسک‌ها متفاوت باشد. در خروجی ما برای breakfast_box تعداد پیکسل مثبت صفر شده (n_pos_pixels=0)، بنابراین Pixel AUROC عملاً قابل محاسبه نیست و NaN می‌شود و AU-PRO هم ۰ می‌ماند. (قابل دفاع است چون مشکل از “نبود GT پیکسلی” است، نه از مدل)



۳- گزارش ایده

ایده را در حوزه‌ی تشخیص ناهنجاری‌های منطقی/ساختاری در تصاویر صنعتی در نظر می‌گیریم (همان چیزی که در LOCO سخت‌تر است) زیرا بر اساس اجرای ما، EfficientAD روی MVTEC AD/bottle تقریباً عالی بوده اما روی MVTEC structural LOCO/breakfast_box افت محسوس دارد. این اختلاف با ماهیت LOCO هم همخوان است، چون LOCO شامل structural anomalies و logical anomalies است (ناهنجاری‌هایی که "ترکیب/چینش/حضور-عدم حضور" اجزا را نقض می‌کنند)، نه فقط لکه و خراش محلی که در خود مقاله EfficientAD هم دقیقاً ادعا می‌کند که برای «logical anomalies» یک جزء global (autoencoder) اضافه می‌کند و نقشه‌ها را کالبره/ترکیب می‌کند.

پس ایده پیشنهادی جدید :

"افزودن «قیود هم‌وقوعی گسسته» روی توکن‌های ویژگی‌ها، برای تقویت تشخیص ناهنجاری‌های منطقی با هزینه محاسباتی بسیار کم یعنی **EfficientAD-DCC** یا **(Discrete Co-occurrence Constraints for Logical Anomalies)**"

مشکل اصلی EfficientAD (طبق تجربه اجرای ما) این است که با وجود جزء سراسری (AE)، هنوز در دیتاستی مثل MVTEC LOCO که ناهنجاری‌های منطقی (مثل کم/زیاد شدن یک جزء، جابه‌جایی یا ترکیب نادرست اجزا) دارد، عملکرد image-level افت می‌کند (در اجرا 84.05 برای breakfast_box در برابر 99.92 برای bottle). دلیل فنی این افت معمولاً این است که اختلاف‌های بازسازی/همسان‌سازی ویژگی (Teacher-Student و AE) بیشتر برای الگوهای بافتی/محلی عالی‌اند، اما برای کشف غیرعادی بودن یک ترکیب از اجزای به‌ظاهر نرمال غلط است پس به یک مدل ساده اما صریح از قیود ترکیبی/هم‌وقوعی نیاز داریم. ایده EfficientAD-DCC دقیقاً همین را اضافه می‌کند: ما ویژگی‌های Teacher را به توکن‌های گسسته تبدیل می‌کنیم و از روی داده‌های نرمال، جدول‌های احتمالی «کدام توکن‌ها در کنار هم و در چه فاصله‌ای مجازند» را یاد می‌گیریم؛ در تست، هر جا ترکیب توکن‌ها «کم‌احتمال/غیرمجاز» باشد، یک امتیاز ناهنجاری منطقی تولید می‌شود و این امتیاز به‌صورت یک نقشه سراسری-منطقی به نقشه‌های EfficientAD اضافه می‌شود. چون این قیود روی نقشه ویژگی کم‌رزولوشن (نه روی تصویر کامل) و با شمارش/هیستوگرام ساخته می‌شود، سربار زمانی بسیار کم است و با هدف EfficientAD (سرعت میلی‌ثانیه‌ای) سازگار می‌ماند.

DCC روی توکن‌ها:

۱. در EfficientAD، Teacher یک feature map می‌دهد (بعد از نرمال‌سازی با mean/std همانند کد).

۲. ما این feature map را به توکن‌های گسسته تبدیل می‌کنیم:

- یا با **K-Means** روی بردارهای ویژگی Teacher (فقط روی تصاویر نرمال train)،
- یا با یک **Vector Quantizer** خیلی سبک (VQ).

نتیجه: برای هر مکان (patch) یک ID توکن داریم.

۳. حالا از روی داده نرمال، یک "مدل قیودی" می‌سازیم:

- برای هر offset مثل (dx,dy) در یک همسایگی کوچک (مثلاً ۸ همسایه یا شعاع ۲)، جدول احتمال/فراوانی می‌سازیم (افزودن قوانین هم‌وقوعی محلی به EfficientAD که خودش دو نقشه محلی و سراسری دارد).

۴. در زمان تست:

- توکن‌ها را استخراج می‌کنیم،
- برای هر patch، امتیاز ناهنجاری منطقی را با منفی لگاریتم احتمال همسایه‌ها می‌گیریم (کم‌احتمال = ناهنجار) که یک نقشه “anomaly منطقی” می‌دهد (روی feature grid) و بعد upsample می‌کنیم تا هم‌سایز تصویر شود.

۵. روش پیشنهادی ما به جای تکیه صرف بر reconstruction، یک “مدل صریح قیود ترکیبی” روی توکن‌های ویژگی Teacher می‌سازد که بیشتر شبیه یک «مدل قواعد/هم‌وقوعی» است و می‌تواند چینش‌های غیرمجاز را با حساسیت بالاتری علامت بزند، بدون اینکه مثل ترنسفورمرهای بزرگ یا VLM‌ها سنگین شود. اگرچه ایده‌های «توکن‌سازی» و «مدل‌سازی قیود/روابط» وجود دارد، اما این ترکیب مشخص DCC سبک روی teacher-token و ادغام مستقیم با EfficientAD برای حفظ ms-latency را به همین فرم رایج نمی‌بینیم؛ با این حال هیچ‌وقت نمی‌شود ۱۰۰٪ تضمین کرد کسی دقیقاً همین را چاپ نکرده باشد!

سایر مدل‌های به‌روز چطور این مشکل را حل کرده‌اند؟

- خانواده Memory Bank و Flow : تمرکز روی پوشش کامل حالت‌های نرمال^۴.
 - به جای یادگیری کامل یک مدل مولد، ویژگی‌های patch-level از داده‌های نرمال را ذخیره می‌کند و در تست، دور بودن از نزدیک‌ترین نمونه‌های نرمال را معیار anomaly قرار می‌دهد. PatchCore با ساخت یک حافظه از patch feature های نرمال و coresets کردن آن، هم دقت بسیار بالا و هم سرعت قابل قبول می‌دهد و روی MVTEC AD نتایج بسیار قوی گزارش می‌کند. در کنار آن، روش‌های flow-based مثل FastFlow تلاش می‌کنند توزیع ویژگی‌ها را با نرمالایزینگ فلو مدل کنند تا هم اطلاعات global و هم local را در یک چارچوب احتمالاتی داشته باشند.
 - نقطه ضعف رایج این خانواده برای LOCO این است که اگر همه اجزا شبیه نرمال باشند ولی ترکیبشان غلط باشد، فاصله‌ی feature ها لزوماً زیاد نمی‌شود؛ برای همین در logical anomalies گاهی نیاز به مدل‌سازی رابطه/قید وجود دارد.

- خانواده Vision-Language و Reasoning : استفاده از دانش عمومی/قواعد برای ناهنجاری منطقی^۵.
 - در سال‌های اخیر، یک خط بسیار فعال، استفاده از Vision-Language Models (VLM/CLIP) برای anomaly detection است. روش‌هایی مثل WinCLIP از CLIP برای صفر-شات/فیو-شات anomaly classification و segmentation استفاده می‌کنند و با اتصال “پرامپت‌های متنی” به تصویر، دانش عمومی

^۴ <https://arxiv.org/pdf/2111.07677>

^۵ <https://arxiv.org/html/2501.01767v1>

را وارد تشخیص می‌کنند. همچنین AnomalyCLIP با prompt learning (به شکل object-agnostic) تلاش می‌کند اثر تفاوت کلاس‌ها را کم کند و روی دیتاست‌های متنوع عملکرد صفر-شات ارائه دهد. برای «ناهنجاری منطقی» به‌طور خاص، کارهای جدیدتری هم به سمت استدلال/قواعد رفته‌اند (مثلاً LogicAD یا LogicQA) که روی LOCO مطرح‌اند و ایده‌شان این است که از داده‌های نرمال، قواعدی استخراج شود یا از متن/حافظه متنی کمک گرفته شود تا تناقض‌های منطقی (کمبود/افزودگی/عدم تطابق) بهتر تشخیص داده شوند.

○ این خانواده معمولاً از نظر محاسباتی سنگین‌تر از EfficientAD هستند، اما در فهم منطقی صحنه مزیت دارند؛ ایده EfficientAD-DCC پیشنهادی ما تلاش می‌کند بخشی از همین توانایی قیدمحور را با هزینه بسیار کمتر وارد EfficientAD کند.

۴- پیاده‌سازی

در EfficientAD، نقشه anomaly عمدتاً از اختلاف ویژگی‌ها (Teacher-Student) و اختلاف شاخه AE ساخته می‌شود؛ اما «فیدهای رابطه‌ای/هم‌وقوعی» بین بخش‌های تصویر به شکل صریح مدل نمی‌شود. ما یک مازول سبک اضافه می‌کنیم که روی ویژگی‌های Teacher (نرمال) یک مدل قیود یاد می‌گیرد:

۱. پیش‌پردازش (یک‌بار برای هر کلاس)

- چند صد تصویر نرمال train را از همان کلاس (مثلاً breakfast_box/train/good) می‌گیریم.
- از معلم، feature map می‌گیریم.
- KMeans با $K=256$ یا ۵۱۲ روی بردارهای ویژگی (نمونه‌برداری شده) می‌زنیم.

۲. ساخت جدول قیود

- برای هر تصویر نرمال:
 - token-id map → Feature
 - برای هر offset محدود (مثلاً ۸ همسایه)، جدول شمارش co-occurrence را پر می‌کنیم.
 - نرمال‌سازی همراه با smoothing (Laplace) تا احتمال صفر نشود.

۳. Inference

- برای test :
 - token map
 - anomaly score از منفی لگاریتم احتمال همسایه‌ها

- upsample و ترکیب با خروجی EfficientAD .

۴. ارزیابی

- Image AUROC : انتظار داریم روی LOCO بهتر شود (چون مشکل منطقی را بهتر پوشش می دهیم).
- Pixel metrics : اگر GT پیکسلی قابل اتکا باشد، AUPRO@0.3 هم می تواند بهتر شود.

فرضیه‌ی اولیه ما

- DCC باید روی LOCO بهتر اثر بگذارد چون LOCO شامل logical anomalies است.
 - روی MVtec AD مثل bottle انتظار داریم تغییر کم باشد (یا کمی بهتر/بدتر)، چون baseline تقریباً سقف زده است.
- علت بهبود احتمالی: DCC ناهنجاری‌های رابطه‌ای را با کم‌احتمال بودن الگوی همسایگی توکن‌ها شکار می‌کند؛ چیزی که اختلاف ویژگی ساده لزوماً به اندازه کافی به آن حساس نیست.
- علت افت احتمالی: اگر DCC بیش‌ازحد وزن بگیرد یا کدبوک بد fit شود، ممکن است noise تولید کند و anomaly-map را خراب کند.

۵- ارائه

ساختار اسلایدها

۱. عنوان پروژه

- مقاله پایه از شاخه ۴

۲. مسئله: Anomaly Detection در صنعت

- چرا بدون داده‌ی معیوب کافی مهم است
- خروجی‌های مورد انتظار: نمره سطح تصویر + نقشه مکانی

۳. EfficientAD در یک نگاه

- Student-Teacher + Global Autoencoder
- ایده‌ی اختلاف ویژگی‌ها برای ساخت anomaly map

۴. اجزای کلیدی معماری

- Teacher فریز، Student و AE قابل آموزش

- Hard feature mining + pretraining penalty

۵. دیتاست‌ها و معیارها

- MVTec AD و MVTec LOCO

- Image AUROC، Pixel AUROC، AU-PRO@0.3

۶. پیاده‌سازی در Colab + TensorBoard

- مراحل: دانلود دیتا، اجرای آموزش/تست، ثبت لاگ‌ها

- خروجی‌ها: weights، anomaly maps، فایل‌های خلاصه

۷. نتایج بازتولید (Baseline)

- نمودار میله‌ای Image AUROC برای دو دیتاست (مثل خروجی‌هایی که گرفتیم)

- تاکید: میانگین روی کلاس‌هایی که اجرا شده‌اند و نه کل benchmark

۸. تحلیل مشکلات و محدودیت‌ها

- دانلود ناقص کل دیتاست‌ها، تفاوت ساختار ماسک‌ها خصوصاً LOCO

- حساسیت به تعداد steps، وزن‌دهی ثابت map_ST و map_AE

۹. ایده جدید پیشنهادی (نوآورانه ولی قابل پیاده‌سازی)

- Adaptive Fusion: وزن‌دهی تطبیقی بین map_ST و map_AE

- هدف: بهتر شدن روی logical anomalies بدون سنگین کردن مدل

۱۰. گام‌های پیاده‌سازی ایده + جمع‌بندی

- تغییر کم در inference، اجرای دوباره، ثبت متریک‌ها در TensorBoard

- [1] https://openaccess.thecvf.com/content/CVPR2023/papers/Liu_SimpleNet_A_Simple_Network_for_Image_Anomaly_Detection_and_Localization_CVPR_2023_paper.pdf
- [2] <https://github.com/DonaldRR/SimpleNet>
- [3] https://openaccess.thecvf.com/content/WACV2024/papers/Batzner_EfficientAD_Accurate_Visual_Anomaly_Detection_at_Millisecond-Level_Latencies_WACV_2024_paper.pdf
- [4] https://anomalib.readthedocs.io/en/v2.0.0/markdown/guides/reference/models/image/efficient_ad.html
- [5] <https://github.com/nelson1425/EfficientAD>
- [6] <https://github.com/search?q=EfficientAD&type=repositories>
- [7] <https://github.com/open-edge-platform/anomalib/discussions/1183>
- [8] <https://github.com/rximg/EfficientAD>
- [9] <https://www.mvtec.com/company/research/datasets/mvtec-loco>
- [10] https://openaccess.thecvf.com/content/CVPR2022/papers/Roth_Towards_Total_Recall_in_Industrial_Anomaly_Detection_CVPR_2022_paper
- [11] https://openaccess.thecvf.com/content/CVPR2023/papers/J Jeong_WinCLIP_Zero-Few-Shot_Anomaly_Classification_and_Segmentation_CVPR_2023_paper.pdf